Instituto Tecnológico de Costa Rica
Escuela de Ingeniería en Computadores
CE1113 Sistemas Empotrados
Prof. Dr.-Ing. Jeferson González Gómez

Taller 7: Modelado de Hardware en QEMU

Diseño de Periféricos MMIO y Co-diseño HW/SW

1. Introducción

En el Taller 06 desarrolló firmware que *usaba* un periférico MMIO ya existente. Este taller invierte el rol: usted es ahora el *diseñador del hardware*. Dado un documento de especificación (este instructivo), implementará el modelo funcional del periférico en QEMU y luego escribirá el firmware que lo valida.
Este flujo refleja fielmente la práctica industrial en el diseño de SoC (*System on Chip*): el equipo de arquitectura define la interfaz del periférico — mapa de registros, semántica, protocolo de handshake — y la publica como especificación. El equipo de firmware comienza a desarrollar sobre el modelo funcional antes de que el RTL (*Register-Transfer Level*) esté completo y mucho antes de que exista silicio [1, 2].

2. Marco Teórico

2.1. El modelo de dispositivo en QEMU

QEMU representa cada periférico como un *objeto* de tipo `SysBusDevice`. Un dispositivo sysbus tiene tres responsabilidades principales:

1. **Región de memoria MMIO**: un rango de direcciones físicas que, cuando el CPU lee o escribe, invoca los callbacks del dispositivo en lugar de acceder a RAM. Se registra con `memory_region_init_io()` y se conecta al bus con `sysbus_init_mmio()`.

2. **Estado interno**: una estructura C (`typedef struct ...State`) con todos los registros y variables del dispositivo. QEMU puede migrar o serializar este estado con el sistema `VMState`.

3. **Lógica de comportamiento**: los callbacks `.read` y `.write` del `MemoryRegionOps` implementan la semántica de cada registro.

2.2. Anatomía del código de un dispositivo QEMU

Un dispositivo QEMU se implementa como un único archivo C compuesto de siete elementos que se explican a continuación, usando el device `riscv-edu-matmul` como ejemplo concreto.

1. Identificador de tipo y macro de acceso

```
/* Cadena que identifica al dispositivo en el sistema de tipos de QEMU
   .
   Es la misma cadena que se usa en qdev_new() desde virt.c. */
#define TYPE_RISCV_EDU_MATMUL "riscv-edu-matmul"

/* Genera automáticamente el cast seguro RISCV_EDU_MATMUL(obj)
   y la declaración forward de RISCVEduMatmulState. */
OBJECT_DECLARE_SIMPLE_TYPE(RISCVEduMatmulState, RISCV_EDU_MATMUL)
```

2. Estructura de estado

Contiene todos los registros del dispositivo tal como existen en hardware. QEMU accede a esta estructura a través del puntero `opaque` que se pasa a los callbacks.

```
typedef struct RISCVEduMatmulState {
    SysBusDevice parent_obj;   /* SIEMPRE el primer campo: herencia en
        C */
    MemoryRegion mmio;         /* Descriptor de la región MMIO */

    uint32_t a[16];    /* Registros de operando A (write-only desde
       firmware) */
    uint32_t b[16];    /* Registros de operando B (write-only desde
       firmware) */
    uint32_t c[16];    /* Registros de resultado  (read-only desde
       firmware)  */
    uint32_t status;   /* Bits DONE / BUSY
                                        */
} RISCVEduMatmulState;
```

3. Callbacks de lectura y escritura MMIO

Son las funciones que QEMU invoca cada vez que el CPU ejecuta una instrucción de carga (`lw`) o almacenamiento (`sw`) sobre la región MMIO del dispositivo. El parámetro `addr` es el *offset* dentro de la región (no la dirección física absoluta).

```
static uint64_t mi_device_read(void *opaque, hwaddr addr, unsigned
   size)
{
    MiDeviceState *s = opaque;   /* recuperar estado desde el puntero
       genérico */
    switch (addr) {
    case REG_STATUS: return s->status;   /* exponer registro de estado
        */
    default:         return 0;
    }
}

static void mi_device_write(void *opaque, hwaddr addr, uint64_t value,
                            unsigned size)
{
    MiDeviceState *s = opaque;
    switch (addr) {
```

```
    case REG_CONTROL:
        if (value & CTRL_START) {
            compute(s);           /* lógica del dispositivo */
            s->status = STATUS_DONE;
        }
        break;
    }
}

/* Estructura que conecta los callbacks con la región MMIO */
static const MemoryRegionOps mi_device_ops = {
    .read      = mi_device_read,
    .write     = mi_device_write,
    .endianness = DEVICE_LITTLE_ENDIAN,
    .valid = {
        .min_access_size = 4,   /* solo accesos word (32-bit) son vá
           lidos */
        .max_access_size = 4,
    },
};
```

4. `instance_init` — constructor de instancia

Se ejecuta una vez por cada instancia del dispositivo al crearlo con `qdev_new()`. Su responsabilidad es inicializar la región MMIO y conectarla al bus. **No** debe establecer valores de registros (eso es responsabilidad de `reset`).

```
static void mi_device_init(Object *obj)
{
    MiDeviceState *s  = MI_DEVICE(obj);    /* cast seguro al tipo
       concreto */
    SysBusDevice  *sbd = SYS_BUS_DEVICE(obj);

    /* Crear la región MMIO asociando los callbacks y su tamaño */
    memory_region_init_io(&s->mmio, obj, &mi_device_ops, s,
                          TYPE_MI_DEVICE, MMIO_SIZE);

    /* Registrar la región en el sysbus (se mapeará luego con
       sysbus_mmio_map) */
    sysbus_init_mmio(sbd, &s->mmio);
}
```

5. `reset` — estado de encendido

QEMU llama a esta función al inicio de la simulación y ante cualquier reset de sistema. Debe llevar *todos* los registros a sus valores de reset definidos en la especificación del hardware.

```
static void mi_device_reset(DeviceState *dev)
{
    MiDeviceState *s = MI_DEVICE(dev);

    /* Todos los registros a su valor de reset (típicamente 0) */
```

```
    memset(s->a, 0, sizeof(s->a));
    memset(s->b, 0, sizeof(s->b));
    memset(s->c, 0, sizeof(s->c));
    s->status = 0;
}
```

6. `class_init` — inicialización de la clase
Se ejecuta *una sola vez* cuando QEMU registra el tipo, no por instancia. Aquí se establece el descriptor textual del dispositivo y se registra la función de reset. La firma exacta de `const void *data` es requerida por la API de QEMU v11.

```
static void mi_device_class_init(ObjectClass *klass, const void *data)
{
    DeviceClass *dc = DEVICE_CLASS(klass);

    dc->desc = "Descripción␣corta␣del␣dispositivo";
    dc->vmsd = &vmstate_mi_device;   /* tabla de campos para migración
        de estado */
    device_class_set_legacy_reset(dc, mi_device_reset);
}
```

7. Registro del tipo y punto de entrada
`TypeInfo` es la tabla que describe el tipo al sistema de objetos de QEMU. `type_init` registra la función `register_types` para que sea invocada automáticamente durante el arranque de QEMU, antes de que se instancie cualquier máquina.

```
static const TypeInfo mi_device_info = {
    .name          = TYPE_MI_DEVICE,      /* identificador de tipo */
    .parent        = TYPE_SYS_BUS_DEVICE, /* herencia */
    .instance_size = sizeof(MiDeviceState),
    .instance_init = mi_device_init,
    .class_init    = mi_device_class_init,
};

static void mi_device_register_types(void)
{
    type_register_static(&mi_device_info);
}

/* Macro que registra la función anterior como inicializador de módulo
   .
   QEMU la invocará antes de construir cualquier máquina virtual. */
type_init(mi_device_register_types);
```

2.3. Ciclo de llamadas en tiempo de ejecución

La Figura 2.3 resume el orden en que QEMU invoca las funciones del dispositivo desde el arranque hasta el acceso MMIO en tiempo de ejecución.

$$\texttt{type_init} \rightarrow \texttt{class_init} \rightarrow \texttt{instance_init} \rightarrow \texttt{reset} \xrightarrow{\text{CPU lw/sw}} \texttt{read / write}$$

Las funciones anteriores al flecha corresponden al arranque de QEMU; `read` y `write` se invocan repetidamente durante la ejecución del firmware cada vez que el CPU accede a la región MMIO.

2.4. Protocolo START / DONE

Un acelerador hardware típico expone un protocolo de handshake de dos señales:

- **START** (escritura del firmware): el firmware indica que los operandos están listos y solicita el cómputo. En hardware real, esto dispara un pipeline o FSM. En el modelo QEMU, el callback de escritura puede ejecutar el cómputo de forma *síncrona* (inmediata) o *asíncrona* (con un `QEMUTimer` para simular latencia).
- **DONE** (lectura del firmware): el firmware sondea el registro de estado hasta que el bit DONE esté activo. En hardware real, este bit lo pone la FSM al terminar. En el modelo QEMU, lo pone el callback de escritura de CONTROL (modo síncrono) o el callback del timer (modo asíncrono).

En este taller se implementará el modo síncrono: el cómputo ocurre instantáneamente dentro del callback de escritura, y el bit DONE se activa antes de que el callback retorne. Esto es suficiente para validar la correctitud funcional del firmware.

2.5. Memory-Mapped I/O: perspectiva del diseñador

Desde el punto de vista del diseñador de hardware, el mapa de registros es un contrato entre el hardware y el firmware. Cada decisión de diseño tiene implicaciones:

- **Ancho de registro**: usar registros de 32 bits (word) aunque el dato sea más pequeño simplifica el alineamiento y es la práctica estándar en periféricos AXI/APB.
- **Acceso**: marcar explícitamente registros como *write-only* (operandos) o *read-only* (resultado, estado) evita usos indebidos y reduce el área de flip-flops en RTL.
- **Semántica de bits**: bits W1C (*write-1-to-clear*) para flags de estado, campos reservados documentados como "escribir 0".
- **Alineamiento**: offsets múltiplos de 4 para garantizar acceso word sin rotación en buses de 32 bits.

3. Dispositivo: Acelerador de Multiplicación de Matrices 4×4

3.1. Descripción funcional

El dispositivo `riscv-edu-matmul` implementa la operación:

$$C = A \times B, \quad A, B \in \mathbb{Z}_{256}^{4\times4}, \quad C_{ij} = \sum_{k=0}^{3} A_{ik} \cdot B_{kj} \in \mathbb{Z}_{2^{32}}$$

Los operandos A y B son matrices de 4×4 elementos de 8 bits sin signo (`uint8_t`). El resultado C almacena cada elemento en un registro de 32 bits (el valor máximo es $4 \times 255^2 = 260\,100$, que cabe en 18 bits pero se expone en 32 para simplicidad).
El dispositivo está instanciado en la máquina `virt` de QEMU con dirección base `0x10011000`.

3.2. Mapa de registros

Offset	Nombre	Acceso	Descripción
`0x00-0x3C`	`A[i][j]`	W	Elemento $[i][j]$ de la matriz A; bits [7:0] válidos
`0x40-0x7C`	`B[i][j]`	W	Elemento $[i][j]$ de la matriz B; mismo layout
`0x80-0xBC`	`C[i][j]`	R	Resultado $C = A \times B$; uint32 por elemento
`0xC0`	`CONTROL`	W	bit 0: START — dispara el cómputo
`0xC4`	`STATUS`	R	bit 0: DONE, bit 1: BUSY

Cuadro 1: Mapa de registros del acelerador `riscv-edu-matmul`.

El índice lineal del elemento $[i][j]$ es $i \times 4 + j$, por lo que su offset es $(i \times 4 + j) \times 4$ sobre la base de la región correspondiente. Por ejemplo, $A[1][2]$ está en `0x10011000 + 0x00 + (1*4+2)*4 = 0x10011018`.

3.3. Protocolo de uso (firmware)

Listing 1: Flujo básico de uso del acelerador desde firmware.

```
/* 1. Cargar operandos */
for (int i = 0; i < 4; i++)
    for (int j = 0; j < 4; j++) {
        MMIO_WRITE(MATMUL_REG_A(i,j), a[i][j]);
        MMIO_WRITE(MATMUL_REG_B(i,j), b[i][j]);
    }

/* 2. Disparar cómputo */
MMIO_WRITE(MATMUL_REG_CONTROL, MATMUL_CTRL_START);

/* 3. Esperar resultado */
while (!(MMIO_READ(MATMUL_REG_STATUS) & MATMUL_STATUS_DONE)) { }

/* 4. Leer resultado */
for (int i = 0; i < 4; i++)
    for (int j = 0; j < 4; j++)
        c[i][j] = MMIO_READ(MATMUL_REG_C(i,j));
```

4. Inicio Rápido

El entorno de desarrollo es idéntico al del Taller 06: todo corre dentro de Docker con la imagen `qemu-tec` que incluye el acelerador `riscv-edu-matmul` en la máquina `virt`.

Paso 1: Construir la imagen Docker

```
docker build --build-arg UID=$(id -u) --build-arg GID=$(id -g) \
             -t taller07 .
```

Este paso clona el fork `qemu-tec`, compila QEMU con los dos dispositivos educativos (`riscv-edu-timer` y `riscv-edu-matmul`) e instala el toolchain RISC-V.

Paso 2: Iniciar el contenedor

```
docker run -it --rm -v "$(pwd)":/workspace taller07
```

Paso 3: Compilar y ejecutar el firmware de validación

```
./scripts/build.sh
./scripts/run.sh
```

Con el esqueleto sin completar, la salida esperada es:

```
=== Taller 07: Acelerador de Matrices 4x4 ===

[HW] Cargando matrices en el acelerador...
[HW] Disparando cómputo (START)...
[HW] Esperando DONE...
```

El firmware quedará colgado en el poll de STATUS porque los TODO no están implementados. Cuando complete los tres TODO, la salida debe terminar con:

```
[TEST] PASS -- todos los elementos coinciden.

=== Fin ===
```

5. Guía de Implementación

5.1. Estructura del repositorio `qemu-tec`

El fork `qemu-tec` es un repositorio Git estándar de QEMU con modificaciones educativas mínimas. Su árbol de directorios relevante es:

```
qemu-tec/
  configure           # script de configuración del sistema de build
  meson.build         # archivo raíz de Meson (no modificar)
  hw/
```

```
    misc/
      meson.build     # <-- (A) registrar nuevos dispositivos miscelá
         neos
      riscv_edu_timer.c     # dispositivo educativo del Taller 06
      riscv_edu_matmul.c    # <-- (B) implementar aquí (copia del
         skeleton)
      [otros devices de QEMU upstream...]
    riscv/
      virt.c          # <-- (C) instanciar el device en la máquina virt
      [otros archivos de la plataforma RISC-V...]
  target/riscv/       # núcleo del emulador RISC-V (no modificar)
  tcg/                # backend de traducción dinámica (no modificar)
```

Para agregar un dispositivo nuevo a QEMU se deben tocar exactamente **tres archivos**. Los tres ya están modificados en `qemu-tec` para incluir `riscv-edu-matmul`; se describen a continuación para que entienda qué hace cada uno.

(A) `hw/misc/meson.build` — registrar el archivo fuente

QEMU usa **Meson** como sistema de construcción. Para que el compilador incluya un nuevo archivo `.c`, debe agregarse una línea en el `meson.build` del subdirectorio correspondiente. Para dispositivos misceláneos de RISC-V, la sección relevante es:

```
# hw/misc/meson.build  (fragmento)
# RISC-V devices
system_ss.add(files('riscv_edu_timer.c'))
system_ss.add(files('riscv_edu_matmul.c'))   # <-- línea agregada
```

`system_ss.add(files(...))` indica a Meson que compile el archivo y lo enlace en el binario `qemu-system-riscv32`. Sin esta línea, el tipo `"riscv-edu-matmul"` no existiría en el binario y `virt.c` fallaría al llamar a `qdev_new()`.
En este taller esta línea ya está agregada. No necesita modificar `meson.build`.

(B) `hw/misc/riscv_edu_matmul.c` — implementación del device

Este es el archivo central del taller: el modelo funcional del acelerador. La plantilla ya está copiada en `qemu-tec/hw/misc/` con los tres TODO sin implementar. Una vez completados los TODO, el script `scripts/rebuild-device.sh` copia el archivo al source de QEMU dentro de la imagen y recompila incrementalmente en ~30 segundos (ver Sección 5.2).

(C) `hw/riscv/virt.c` — instanciar el device en la máquina

La máquina `virt` de RISC-V se define en `hw/riscv/virt.c`. Este archivo construye la plataforma completa: RAM, UART, PLIC, CLINT, RTC, y los dispositivos educativos. Para que un device registrado en `meson.build` sea accesible al firmware, debe instanciarse aquí con tres llamadas:

```
/* hw/riscv/virt.c  (fragmento -- ya incluido en qemu-tec) */

/* Paso 1: crear la instancia del objeto device */
DeviceState  *edu_matmul     = qdev_new("riscv-edu-matmul");
```

```
SysBusDevice *edu_matmul_sbd = SYS_BUS_DEVICE(edu_matmul);

/* Paso 2: realizar el device (ejecuta instance_init y reset) */
sysbus_realize_and_unref(edu_matmul_sbd, &error_fatal);

/* Paso 3: mapear la región MMIO a la dirección física 0x10011000 */
sysbus_mmio_map(edu_matmul_sbd, 0, 0x10011000);
```

El segundo argumento de `sysbus_mmio_map` es el índice de la región (0 = primera región, correspondiente a la registrada con `sysbus_init_mmio` en `instance_init`). El tercer argumento es la dirección base física que verá el firmware.
En este taller estas tres líneas ya están agregadas. No necesita modificar `virt.c`.

5.2. Flujo completo de desarrollo

La imagen Docker conserva el código fuente de QEMU en `/opt/qemu-tec/`, lo que permite recompilar solo el archivo modificado sin reconstruir la imagen completa. El ciclo de desarrollo es:

1. Editar `src/device-skeleton/riscv_edu_matmul.c` (completar TODO).
2. Dentro del contenedor, aplicar los cambios a QEMU y recompilar (~30 segundos):

   ```
   ./scripts/rebuild-device.sh
   ```

   Este script copia el skeleton a `/opt/qemu-tec/hw/misc/`, ejecuta `ninja` de forma incremental y actualiza el binario.
3. Compilar y ejecutar el firmware de validación:

   ```
   ./scripts/build.sh && ./scripts/run.sh
   ```

4. Repetir desde el paso 1 hasta obtener `[TEST] PASS`.

> **Tiempos de compilación:** `docker build` (primera vez) tarda ~15 minutos y solo se ejecuta una vez. Las iteraciones posteriores con `rebuild-device.sh` tardan ~30 segundos porque Ninja recompila únicamente el archivo modificado y reenlaza el binario.

5.3. TODO 1 — Implementar `matmul_compute()`

Esta función debe calcular $C = A \times B$ usando los arrays `s->a`, `s->b` y `s->c` del estado del dispositivo.
El índice lineal del elemento $[i][j]$ es `i * MAT_N + j`. Los elementos de `s->a` y `s->b` son `uint32_t` que contienen un valor `uint8_t` en los bits [7:0]. El resultado en `s->c` debe ser `uint32_t`.

Listing 2: Estructura del triple loop para multiplicación de matrices.

```
for (uint32_t i = 0; i < MAT_N; i++) {
    for (uint32_t j = 0; j < MAT_N; j++) {
```

```
            s->c[i * MAT_N + j] = 0;
            for (uint32_t k = 0; k < MAT_N; k++) {
                /* Completar: acumular a[i][k] * b[k][j] */
            }
        }
}
```

5.4. TODO 2 — Disparar el cómputo al recibir START

En el handler de escritura, dentro del `case REG_CONTROL`, debe:

1. Verificar si el bit `CTRL_START` está activo en el valor escrito.
2. Si es así, marcar el dispositivo como `BUSY`.
3. Llamar a `matmul_compute(s)`.
4. Marcar el dispositivo como `DONE`.

5.5. TODO 3 — Exponer STATUS y el resultado al firmware

En el handler de lectura debe:

- Para el rango `REG_C_BASE` a `REG_CONTROL`: calcular el índice `idx = (addr - REG_C_BASE) / 4` y devolver `s->c[idx]`.
- Para `REG_STATUS`: devolver `s->status`.

6. Evaluación

Parte 1 — Implementación del device (50 %)

Complete los tres TODO en `riscv_edu_matmul.c` de manera que el firmware de validación produzca la salida `[TEST] PASS`.

R1. `matmul_compute()` calcula correctamente $C = A \times B$.

R2. El bit DONE se activa en STATUS tras escribir START en CONTROL.

R3. La lectura de los registros C devuelve los valores correctos.

R4. La salida del firmware es `[TEST] PASS` para la matriz de prueba.

R5. El código compila sin advertencias con `-Wall`.

Parte 2 — Segunda matriz de prueba (20 %)

Agregue en `app.c` un segundo caso de prueba con matrices de su elección (al menos un elemento distinto de cero por fila y columna). Calcule el resultado esperado a mano o con una herramienta externa, muéstrelo en el informe y verifique que el firmware produzca `PASS`.

Parte 3 — Análisis y extensión (30 %)

Escoja **una** de las siguientes opciones:

Opción A — Latencia simulada:

Modifique el dispositivo para simular latencia de hardware usando un `QEMUTimer`: al recibir START, marque BUSY y programe el timer para que ejecute `matmul_compute()` y active DONE después de N microsegundos (elija N entre 100 y 10 000). En el firmware, mida el tiempo de espera leyendo el contador del timer educativo e incluya la medición en el informe.

Opción B — Dimensión parametrizable:

Generalice el dispositivo para soportar matrices de 2×2, 3×3 y 4×4, seleccionables mediante un registro `DIM` en offset `0xC8`. Actualice el firmware para probar las tres dimensiones.

Opción C — Saturación aritmética:

Modifique el tipo de resultado a `uint8_t` (resultado saturado a 255 si el acumulador supera 255). Diseñe una matriz de prueba donde la saturación sea observable y demuéstrela en el firmware.

Entregable

- Archivo `ApellidoNombre_Taller07.tar.gz` con:
  1. `src/device-skeleton/riscv_edu_matmul.c` (con los TODO completos).
  2. `src/firmware/app.c` (con los casos de prueba de Partes 1 y 2).
  3. `README.txt`: nombre, carné, opción elegida en Parte 3, instrucciones para compilar y ejecutar.
- Informe PDF (máximo 3 páginas) con:
  1. Descripción de la implementación de `matmul_compute()`.
  2. Captura de la salida serial mostrando PASS para ambos casos de prueba.
  3. Descripción e implementación de la opción elegida en Parte 3.

Referencias

[1] F. Bellard, "QEMU, a Fast and Portable Dynamic Translator," in *Proc. USENIX Annual Technical Conference (FREENIX Track)*, pp. 41–46, 2005.

[2] ARM Limited, *AMBA APB Protocol Specification*, IHI0024C, disponible en: `https://developer.arm.com/documentation/ihi0024`, 2010.